Jean Biche

Informatique

Sale

I.

Q1. Oui / Non / Non

il faut exhiber le cycle

Q2. Soit $(s_0, s_1, \dots, s_m)$ un cycle eulérien avec $s_0 = s_m$.

Faux pour s_0

Alors pour tout $s \in S$, le degré de s est deux fois le nombre d'occurrences de s dans le cycle, car chaque occurrence correspond à deux arêtes (une avant et une après).

Donc deg(s) est pair.

Q3.

```
bool is_eulerian(){
    int d=0;
    for (int i=0; i<N; i++){
        for (int j=0; j<N; j++){
            if (graph[i][j] != NULL){
                d++;
            }
        }
    }
    if (d%2 != 0) return false;
}
return true; }
```

Même si le code est correct, il aurait été plus transparent de mettre cette variable dans la première boucle (ou de faire une remarque)

Q4. Soit $v_0 \in S$. Comme G est connexe et $N \geq 2$, $\deg(v_0) \geq 1$.

Comme $\deg(v_0)$ est pair, $\deg(v_0) \geq 2$.

On pose v_1 un voisin de v_0. Pour $i \geq 1$, on pose v_{i+1} un voisin de v_i différent de v_{i-1}.

Un tel sommet existe, car comme pour v_0, $\deg(v_i) \geq 2$.

Par le principe des tiroirs, il existe $0 \leq i < j \leq N$ tels que $v_i = v_j$.

Le cycle ne contient pas nécessairement v_0 !

Le chemin $\underline{(v_i, v_{i+1}, \dots, v_j)}$ est bien un cycle.

Q5.

```
edge* round_trip(int start){
    edge* e = any_edge_from(start);
    while (e.dst != start){
        connect(e, any_edge_from(e.dst));
        e = e.next;
    }
    return e;
}
```

distingue → et.

Le cycle n'est pas refermé

Q6.

```
edge* find_vertex_with_edges(edge* c){
    edge* cur = c;
    while (cur != c){
        if (has_edges(cur.src)){
            return cur;
        }
```

La boucle termine tout de suite !

```
        cur = cur->next;
    }
    return NULL;
}
```

Q7

```
edge* eulerian_cycle(int start){
    edge* cycle = NULL;
    while (true){
        edge* e = round_trip(start);
        join(e, cycle);
        e = find_vertex_with_edges(cycle);
        if (e != NULL)
            start = e->src;
        else
            return cycle;
    }
```

non défini si l'un des deux est NULL

Q8.

round_trip : en $O(N)$ car le cycle est de taille $\leq N$

find_vertex_with_edges : en $O(E)$ car il faut parcourir tout le cycle.

has_edge et any_edge_from ne sont pas en $O(1)$

eulerian_cycle : la boucle est de taille $\leq E$, donc $O(E \times (E+N))$

Sale !

~~On ne peut pas faire mieux~~

Très vague et n'a aucun sens ici

On peut faire $O(N+E)$ avec un parcours de graphe.

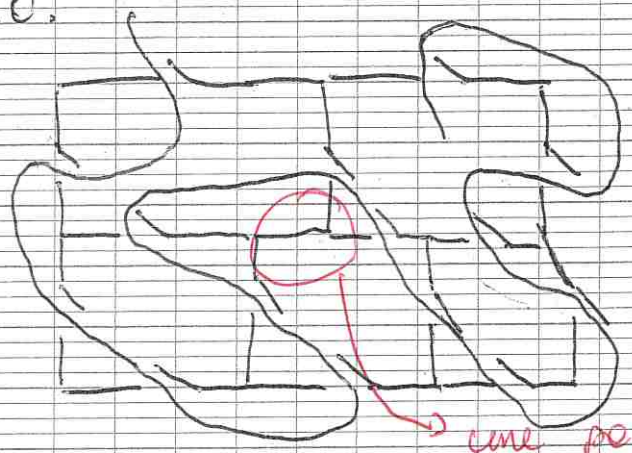
Q9. Sens direct : comme Q2, sauf pour les extrémités.

Trop succint, on attend plus de détails

Sens réciproque : comme Hierholzer en partant d'un sommet de degré impair.

insuffisant, tu peux ne pas avoir de cycle initial

Q10.



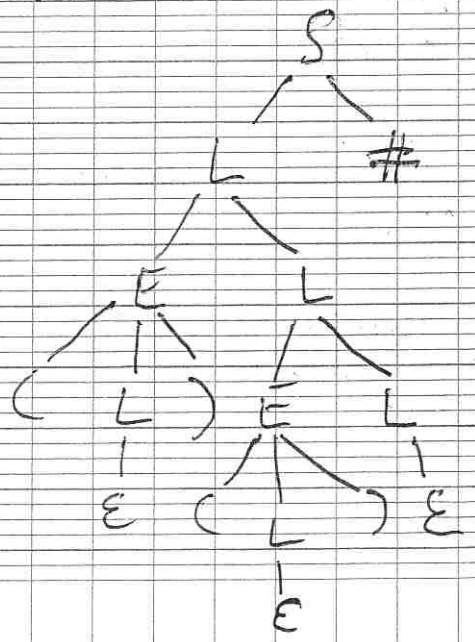
Oui.

une porte a disparu.

II.

On demande un résultat pour tout n, pas pour un n particulier !

Q11. Pour $n = 5$, le mot $()()\#$ est engendré par :



il fallait la dérivation gauche, par l'arbre de dérivation.

Jean Biche

illisible

Q12. rec

```
let parseL = function
  | (Rpar | Eof) -> Epsilon
  | l -> parseE (parseL l)
```

je suppose qu'il est défini

et la queue de la liste ?

cela n'a aucun sens !

```
let parseE = function
  | Symm -> Symm
  | Rpar :: l ->
      let q = parseL l in
      List.tl q   (1)
  | _ -> raise SyntaxError
```

pas une liste

on veut renvoyer une liste

L

il faut vérifier que la tête est Rpar

(1) Pour enlever la parenthèse fermante.

Q13.

```
let accepte l =
  parseS l = []
```

il faut gérer le cas de la levée d'exception.

Q14.

Il n'y a que L, car c'est le seul avec une règle $L \to \varepsilon$

Bonne réponse, mais pour la mauvaise raison.

Q15.

Le nombre de X tels que NUL(X) vaut false est un ~~invariant~~ de boucle, donc l'algorithme termine. variant.

Q16. Je ne suis pas sûr de comprendre la question... C'est pas définition de l'algorithme ?

La définition est mal comprise, on attend une preuve par récurrence, dans les 2 sens

J'essaie quand même :

À l'étape 2, s'il y a une règle $X \to \varepsilon$, on pose NUL(X) à true, donc l'algorithme trouve bien la bonne valeur de NUL(X).

Q17.

	S	L	E
PREMIERS	~~L~~	~~ε, E~~	symb, (
SUIVANTS	#	~~L~~	~~L~~

La table ne contient que des éléments de T

Q18. On a juste remplacé $L \to EL$ par $L \to LE$.

Trop vague, on attend une induction

Vu que L et E engendrent toujours les mêmes langages, que $L = E^*$, le sens de concaténation ne change pas les mots obtenus.

Q19

	symb	(	)	#
S'	L'#	L'#		L'#
L'	L'E'	L'E'	ε	ε
E'	symb	(L')		

ou ε

On peut faire l'analyse comme avant, en inversant l'ordre des appels à parseL et parseE.

III.

Q20.

```
let rec check n = function
  | True -> true
  | False -> ~~false~~ true
  | If(i, g, d) ->
      let rec aux = function
        | If(j, g, d) when j i -> false
        | If(_, g, d) -> aux g && aux d
        | _ -> true
      in
      aux g && aux d && i < n
```

sale (et mauvaise inégalité)

Pas la bonne complexité

sale

Q21.

If(0, If(1, true, false), If(2, false, true))

on inverse car 0 vaut false.

Peut valoir true pour
0 -> true
1 -> true
2 -> false

Q22. On a n variables, mais il n'est pas précisé si elles sont entre 0 et n-1, donc il faut renumérotter...

si ! c'est indiqué en début de partie !

```
let renumerote n f =
  let cur = ref 0 in
  let dico = Hashtbl.create () in
```

il faut un argument entier

Que fait cette fonction ?

```
let rec aux = function
    | True -> True
    | False -> False
    | If(i, g, d) ->
        if !(Hashtbl.mem dico i) then
            Hashtbl.add dico i cur; !cur
            incr cur;
        If(Hashtbl.find dico i, aux g, aux d)
in
aux f;;
```

pas la négation en OCaml

Maintenant on peut faire du backtracking.

```
let decide n f =
    let tab = Array.make n true in
    let rec aux = function
        | True -> Tautology
        | False -> Refutation of tab
        | If(i, g, d) ->
            if aux g = Tautology then
                tab.(i) <- false;
                aux d
            else Refutation of tab
in
aux (renumeroter n f)
```

Explique ce que fait aux

la case n'est jamais remise à true

Jean Biche

Q23.

```
let rec mk_not = function
  | True -> False
  | False -> True
  | If (i, g, d) -> If (i, mk_not d, mk_not g)
```

non ! il ne faut pas inverser

on inverse car on prend la négation.

Q24.

```
let rec mk_or f g = match f, g with
  | True, g -> True
  | False, g -> False
  | If(i, fg, fd), If(i, gg, gd) ->
      If(i, mk_or fg gg, mk_or fd gd)
  | _ -> mk_or g f
```

erreur de filtrage. L'indice de variable n'est pas forcément le même

La complexité est linéaire.) en quoi ?) explique la raison de ce cas

Q25. On veut $\neg(F_1 \wedge F_2 \wedge F_3 \wedge F_4 \wedge F_5 \wedge F_6)$.
On utilise de Morgan.

Détaille un peu, on veut une construction des différentes formules, et explicite la fonction mk_and si besoin.